Reinforcement Learning
Lecture V: Value Iteration

Asad Jeewa
University of Cape Town



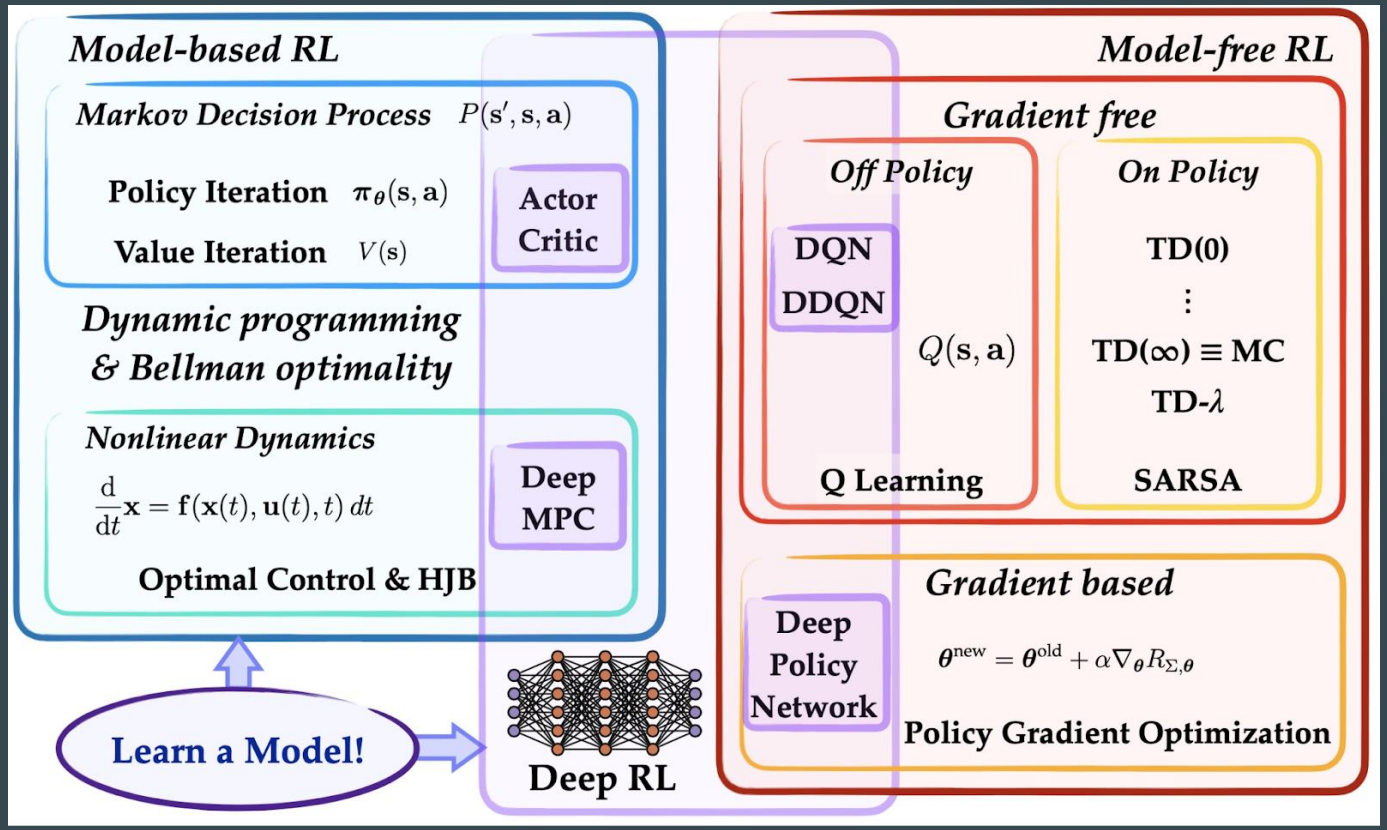
What will be covered

- How do we learn the optimal value function?
- Dynamic Programming
- Coding Value Iteration

Recap

- MDP = $< S, A, T, R, \gamma >$
- Value function $V^{\pi}(s)$ = expected return following policy π
- Optimal value $V^{*}(s) = \max_{\pi} V^{\pi}(s)$
- Goal: find π^{*} that maximises cumulative reward

The Bigger Picture



Recall: Model-based RL

- Given: MDP $(S, T, A, R, \boldsymbol{\gamma})$
- Goal: Find an optimal policy: π^*
- How: Exact Methods find the true optimal solution using the known model (T, R).
  - **Value iteration**
  - Policy iteration

Value Iteration

- What does it mean to solve an MDP?
- Goal: Find the optimal policy π^* that maximises expected return.

1. Initialise $V(s)$ or $Q(s,a)$ by solving the Bellman optimality equations.
2. **Iteratively update the values using the Bellman optimality equation until they converge.**
3. Derive the optimal policy π^* by choosing actions that maximise the converged values.

Value Iteration

- Method for finding the optimal value function V^* for an **MDP**.
- Assumes a finite (discrete) state and action space.
- Model-Based Algorithm: the transition dynamics T and reward function R are assumed to be known
- Dynamic programming (DP) algorithm.

Value Iteration and Bellman Optimality

- The optimal value function satisfies the Bellman Optimality Equation.
- The Bellman equation is a top-down recursive definition.
- Value iteration is a bottom-up algorithm used to solve it.
  - Computes values iteratively from terminal states upwards

Dynamic Programming

- Dynamic Programming solves a problem by tackling smaller, overlapping subproblems and combining their optimal solutions.
- In MDPs, these subproblems are the values of successor states: each state's value depends on the values of states it can reach.
- Bellman equations capture this recursive relationship

Value Iteration: Repeatedly applying the Bellman update to compute V^*

Value Iteration Algorithm (Async)

Value Iteration, for estimating $\pi \approx \pi_*$

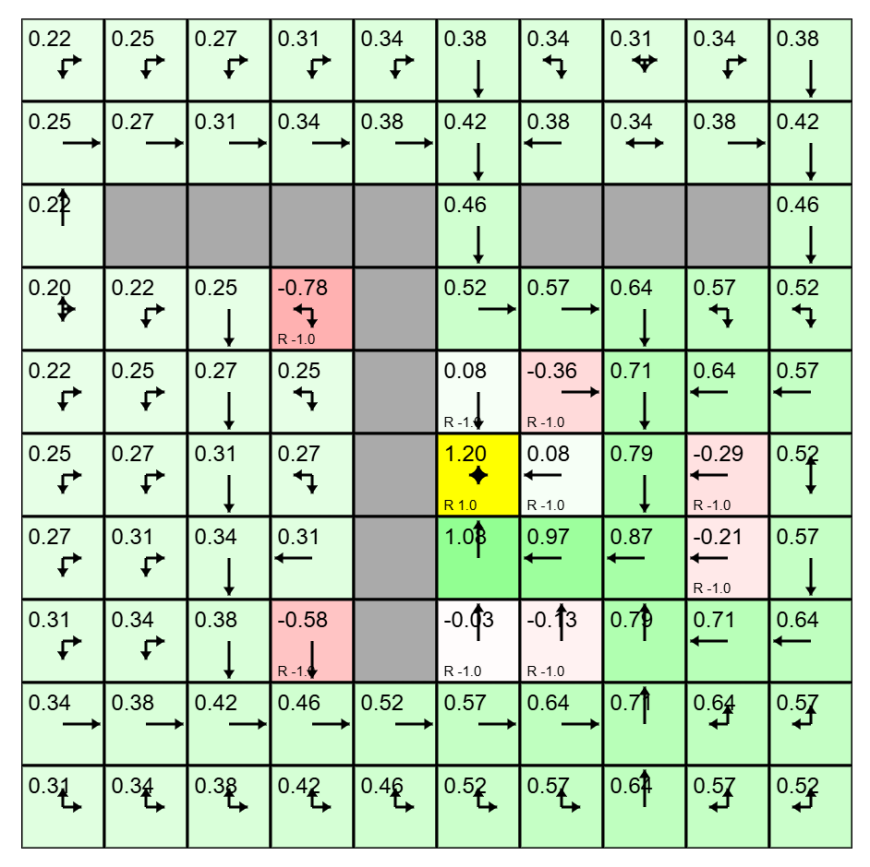
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$

https://medium.com/@numsmt2/reinforcement-learning-chapter-4-dynamic-programming-part-3-value-iteration-6f01f6347813

Value Iteration Visually



https://cs.stanford.edu/people/karpathy/reinforcejs/gridworld_dp.html

Value Iteration Update Schemes

- Synchronous
  - Update all states using values from the previous iteration
  - Consistency and parallelisation
- Asynchronous
  - Update states one at a time using the latest available values
  - Typically results in faster convergence.

Synchronous value iteration computes updates using a separate value table, while asynchronous value iteration updates the value function in place.

What about Complexity?

- What is the Big O Complexity of Value Iteration?
- $O(|S|^2|A|)$: One loop over states × one loop over actions × one loop over next states
- This assumes a stochastic environment and policy

Challenges

- Requires a perfect model of the environment (transition T and reward R).
- Computationally expensive for large state/action spaces (curse of dimensionality).

Value Iteration Convergence

- Guaranteed due to the contraction property of the Bellman optimality operator.
- $\| \mathcal{T}V - \mathcal{T}U \|_\infty \leq \gamma \| V - U \|_\infty, 0 \leq \gamma < 1$
- $\mathcal{T}$ is the Bellman operator
- $V_{k+1}(s) = \mathcal{T}V_k(s) = \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$
- NB: k represents iterations not timesteps!

Contraction of Bellman operator proof

https://www.tuananhle.co.uk/notes/bellman-optimality.html

Claim 1. The Bellman optimality operator $\mathcal{B}^\star$ is a contraction mapping. That is

$$\|\mathcal{B}^\star[Q] - \mathcal{B}^\star[U]\|_\infty \leq \alpha \|Q - U\|_\infty \tag{6}$$

where Q and U are functions mapping from a state-action pair (s, a) to a real number, $\alpha \in [0, 1)$ is a constant and $\|f\|_\infty = \max_x f(x)$ is the L-infinity norm (https://en.wikipedia.org/wiki/L-infinity) of a function $f(x)$.

To prove this, we'll make use of the following identity.

Claim 2. For any two real-valued functions $f(x), g(x)$,
$\max_x f(x) - \max_x g(x) \leq \max_x (f(x) - g(x))$.

Proof of Claim 2. Let $x^\star = \text{argmax}_x f(x)$. Then

$$\begin{aligned}
\text{LHS} &= f(x^\star) - \max_x g(x) && (7)\\
&\leq f(x^\star) - g(x^\star) && (8)\\
&\leq \text{RHS}. && (9)
\end{aligned}$$

□

Proof of Claim 1. To show (6),

$$\begin{aligned}
\text{LHS} &= \|(\text{reward term}) + (Q \text{ term}) - (\text{reward term}) - (U \text{ term})\|_\infty && \text{(sub in (1))} && (10)\\
&= \left\| \gamma \mathbb{E}_{p_{\text{env}}(s'|s,a)} \left[\left(\max_{a'} Q(s', a') \right) - \left(\max_{a'} U(s', a') \right) \right] \right\|_\infty && \text{(cancel reward terms)} && (11)\\
&\leq \left\| \gamma \mathbb{E}_{p_{\text{env}}(s'|s,a)} \left[\max_{a'} \left(Q(s', a') - U(s', a') \right) \right] \right\|_\infty && \text{(use Claim 2)} && (12)\\
&\leq \left\| \gamma \mathbb{E}_{p_{\text{env}}(s'|s,a)} \left[\max_{s''} \max_{a'} \left(Q(s'', a') - U(s'', a') \right) \right] \right\|_\infty && (\max_x f(x) \geq f(x') \text{ for all } x') && (13)\\
&= \left\| \gamma \mathbb{E}_{p_{\text{env}}(s'|s,a)} \left[\max_{(s'', a')} \left(Q(s'', a') - U(s'', a') \right) \right] \right\|_\infty && (\max_x \max_y f(x, y) = \max_{(x,y)} f(x, y)) && (14)\\
&= \left\| \gamma \mathbb{E}_{p_{\text{env}}(s'|s,a)} \left[\|Q - U\|_\infty \right] \right\|_\infty && \text{(definition of the norm)} && (15)\\
&= \left\| \gamma \|Q - U\|_\infty \right\|_\infty && \text{(expectation of a constant function is constant)} && (16)\\
&= \gamma \|Q - U\|_\infty && \text{(norm of a constant function is constant)} && (17)\\
&= \text{RHS}, && && (18)
\end{aligned}$$

Notes on the proof

- $\|\cdot\|_\infty$ simply means look at every possible pair and find the one where the functions disagree the most.
- Notation difference:
  - $\mathcal{B}^* = \mathcal{T}$
  - Q is used instead of V
- LHS $= \|\mathcal{B}^*[\mathrm{Q}] - \mathcal{B}^*[\mathrm{U}]\|_\infty$
  - $\mathcal{B}^*[\mathrm{Q}] = \sum_{s',r} p(s', r|s, a)[r + \gamma \max_{a'} Q(s', a')]$
- Trick: Replacing s' with the "global maximum" over all s''a'' provides a fixed upper bound that no longer depends on the transition probability distribution.

Resources

- Shared RL code space
- Sutton & Barto (1998), Reinforcement Learning: An Introduction, Chapter 4.
- gibberblot.github.io/rl-notes/single-agent/value-iteration